

EASA COLLEGE
OF ENGINEERING & TECHNOLOGY (ECET)
APPROVED BY AICTE, NEW DELHI | AFFILIATED TO ANNA UNIVERSITY, CHENNAI
AN AUTONOMOUS INSTITUTION



SURVEY OF ENSEMBLE LEARNING TECHNIQUES FOR TEXT CLASSIFICATION

BAGGING • BOOSTING • VOTING CLASSIFIERS

A MINI PROJECT REPORT

Submitted by

YOGADHARSHINI

Register No: E720524AM037

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

EASA COLLEGE OF ENGINEERING & TECHNOLOGY

ANNA UNIVERSITY : CHENNAI 600 025

OCTOBER 2026

ANNA UNIVERSITY :: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this mini project report "SURVEY OF ENSEMBLE LEARNING TECHNIQUES FOR TEXT CLASSIFICATION" is the bonafide work of "YOGADHARSHINI" who carried out the mini project work under my supervision.

SIGNATURE

DR. M G DINESH
HEAD OF THE DEPARTMENT
Department of Artificial Intelligence and Data Science
EASA COLLEGE OF ENGG & TECH,
COIMBATORE - 641 105

SIGNATURE

MR. C G KIRUBHANANDHAN
ASSISTANT PROFESSOR
Department of Artificial Intelligence and Data Science
EASA COLLEGE OF ENGG & TECH,
COIMBATORE - 641 105

ABSTRACT

Text classification assigns documents, sentences, reviews or messages to predefined categories. It supports sentiment analysis, spam filtering, topic classification, intent detection and customer-support routing. Because text is unstructured, traditional machine-learning systems first convert it into numerical features such as Bag of Words or Term Frequency–Inverse Document Frequency (TF-IDF).

Ensemble learning combines multiple learners to produce a joint prediction. This mini project surveys and demonstrates three widely used approaches: **bagging**, **boosting** and **voting classifiers**. Bagging trains models on bootstrap samples and aggregates their predictions; boosting trains learners sequentially and places more emphasis on examples that earlier learners classified incorrectly; voting combines the outputs of different models using either class labels or class probabilities.

The project design uses TF-IDF features and compares a bagging classifier, AdaBoost, a soft-voting classifier and a Logistic Regression baseline on a binary sentiment task. Accuracy, precision, recall, F1-score and confusion matrices are used for evaluation. The accompanying Python notebook demonstrates a reproducible end-to-end workflow and produces comparison tables and plots.

The implementation dataset is a small generated teaching dataset. It is useful for checking that the code runs, but its scores must not be interpreted as real-world benchmark results. A rigorous study must use a public dataset, protect the test set from leakage, tune models only on training or validation data and report limitations clearly. The report also explains the theory behind each method, worked numerical examples, architecture, evaluation practice, applications, challenges and future work.

TABLE OF CONTENTS

1. INTRODUCTION 7

2. PROBLEM STATEMENT AND MOTIVATION 8

3. AIM, OBJECTIVES AND SCOPE 8

4. FUNDAMENTALS OF TEXT CLASSIFICATION 9

5. PREPROCESSING AND TF-IDF 9

6. ENSEMBLE LEARNING PRINCIPLES 10

7. BAGGING 11

8. BOOSTING AND ADABOOST 12

9. VOTING CLASSIFIERS 13

10. COMPARISON OF ENSEMBLE METHODS 14

11. SYSTEM ARCHITECTURE 15

12. DATASET AND EXPERIMENTAL DESIGN 16

13. IMPLEMENTATION DETAILS 17

14. EVALUATION METRICS 18

15. RESULTS AND INTERPRETATION 19

16. REAL-WORLD APPLICABILITY 20

17. CHALLENGES AND LIMITATIONS 20

18. FUTURE ENHANCEMENTS 21

19. CONCLUSION 21

20. DEVELOPMENT METHODOLOGY 22

21. FEATURES OF THE PROJECT 23

REFERENCES 23

APPENDIX A – PROGRAM SOURCE CODE 24

APPENDIX B – REPRODUCIBILITY CHECKLIST AND VIVA QUESTIONS 25

APPENDIX C – LIST OF SYMBOLS AND ABBREVIATIONS 25

LIST OF TABLES

Table 4.1 Quick reference – text representation methods 9

Table 5.1 Illustrative IDF calculation for N = 3 documents 10

Table 9.1 Summary comparison of bagging, boosting and voting 14

Table 10.1 Practical trade-offs between the three ensemble strategies 14

Table 11.1 Suggested public datasets for a rigorous extension 16

Table 12.1 Experimental protocol used in the project 17

Table 12.2 Typical hyperparameter search ranges 17

Table 13.1 Model configuration used in the demonstration 18

Table 14.1 Metric formulas and interpretation 18

Table 14.2 Confusion matrix layout with the illustrative counts 18

Table 15.1 Result of the demonstration run (generated dataset) 19

Table 20.1 Methodology phases and activities 22

Table 21.1 Project features summary 23

LIST OF FIGURES

Figure 7.1 Conceptual comparison of bagging, boosting and voting 12

Figure 8.1 AdaBoost weight update for the worked example ($\varepsilon = 0.20$, $\alpha \approx 0.693$) 13

Figure 11.1 Overall system flow of the ensemble text-classification project 15

1. INTRODUCTION

1.1 About the Project

Digital communication creates very large volumes of reviews, e-mails, social-media posts, news articles and customer-support messages. Manual categorisation of such text is slow, expensive and inconsistent between human annotators. Text classification automates this process by learning patterns that connect the content of a text to a label. A trained classifier can assign sentiment, detect spam, identify the topic of an article or route a request to the appropriate department within milliseconds.

A typical workflow includes collecting labelled examples, cleaning and normalising the text, splitting the data, extracting features, training models, evaluating predictions and finally deploying a selected system. Every stage affects performance. Incorrect labels, duplicated documents, imbalanced classes, vocabulary mismatch and careless evaluation can all produce misleading results, so a disciplined experimental design is as important as the choice of algorithm.

Traditional text classification relies on sparse lexical features and algorithms such as Naive Bayes, Logistic Regression, Support Vector Machines and decision trees. These models have different assumptions and therefore tend to make different errors. Ensemble learning takes advantage of this fact by combining several learners instead of relying on one model alone.

This mini project, **Survey of Ensemble Learning Techniques for Text Classification**, focuses on bagging, boosting and voting. The study uses the same TF-IDF feature design and the same train/test split for every approach so that the comparison is meaningful. A single-model baseline is included because a complex ensemble should demonstrate a useful benefit before it is preferred over a simple model.

1.2 Background

The idea of combining predictors is old and intuitive: a committee of reasonably competent members that make partly independent mistakes is usually more reliable than any single member. In machine learning this intuition was formalised in the 1990s through bootstrap aggregating (bagging) by Breiman and through boosting algorithms such as AdaBoost by Freund and Schapire. Voting and stacking schemes extended the idea to heterogeneous model families.

Text classification is a natural testing ground for ensembles. Text features are high-dimensional and sparse, small changes in the training sample can change which words a model relies on, and different algorithm families exploit lexical evidence in different ways. At the same time, strong linear models are already very effective on sparse TF-IDF features, which makes it essential to measure whether the additional complexity of an ensemble produces a genuine gain.

1.3 Scope of the Mini Project

The scope covers a literature-based survey of the three ensemble families, the design of a reproducible TF-IDF pipeline, the implementation of four classifiers in scikit-learn, the computation of standard evaluation metrics and a discussion of limitations. The practical work is restricted to binary sentiment classification with traditional machine-learning models. Deep neural networks and large pretrained language models are discussed only as future extensions.

2. PROBLEM STATEMENT AND MOTIVATION

The problem addressed is how to apply and compare bagging, boosting and voting classifiers for text classification. Text can be ambiguous, sparse, noisy and domain-dependent. One learner may overfit a small dataset, overlook important patterns or perform poorly on minority classes. Ensembles may reduce some of these problems, but they can also increase training time, memory use and model complexity.

Formally, given a labelled dataset $D = \{(x_i, y_i)\}$, $i = 1, \ldots, n$, where x_i is a text document and y_i is its known class, the goal is to learn a function f that predicts a label for unseen text. The experiment compares several ensemble methods against a baseline under a consistent feature-extraction pipeline, so that any observed difference can be attributed to the learning strategy and not to a change in data handling.

The motivation is practical as well as academic. Automated text classification is used in customer-feedback analysis, e-mail filtering, news organisation, help-desk routing and many other applications. Understanding the trade-offs between ensemble methods helps a practitioner select a model based on predictive performance, interpretability, resource use and the cost of different kinds of errors.

The report is guided by the following research questions:

- How do the three ensemble strategies differ in the way they create, train and combine their member models?
- What are their advantages and limitations when the features are sparse TF-IDF vectors?
- Which evaluation measures are necessary beyond accuracy?
- What experimental controls are required before claiming that one approach is better than another?

3. AIM, OBJECTIVES AND SCOPE

The aim of the project is to survey and demonstrate the applicability of bagging, boosting and voting classifiers to text classification using a consistent TF-IDF representation and a transparent evaluation workflow.

The objectives of the mini project are:

- To explain the principles of ensemble learning, including bias, variance, diversity and aggregation.
- To describe the algorithms and workflows of bagging, AdaBoost and hard and soft voting.
- To convert raw text into TF-IDF features without leaking information from the test set.
- To implement bagging, AdaBoost and soft voting and compare them with a Logistic Regression baseline.
- To calculate accuracy, precision, recall, F1-score and confusion matrices.
- To discuss limitations, ethical considerations and possible future extensions.

The practical scope is binary sentiment classification with positive and negative labels. The report concentrates on traditional machine-learning models and sparse lexical features. It does not claim to cover every natural-language-processing task, nor does it provide a production-ready system for high-impact decisions.

The expected outcome is a working notebook, a documented methodology, a model comparison and a clear explanation of what the results mean. The project also highlights that an ensemble is not automatically superior: a simpler model may be faster, easier to interpret and just as effective on a particular dataset.

4. FUNDAMENTALS OF TEXT CLASSIFICATION

Text classification is the assignment of a category to a unit of text based on its content. In supervised learning, the training set contains documents paired with known labels. A model learns a relationship between text features and classes and then predicts labels for new, unseen examples.

Binary classification selects one of two labels, such as spam or legitimate. **Multiclass classification** selects exactly one label from three or more categories, such as sports, business or science. **Multilabel classification** allows several labels for a single document, while **hierarchical classification** organises labels into a tree of categories and sub-categories.

Common algorithms include Naive Bayes, Logistic Regression, linear Support Vector Machines, decision trees and Random Forest. Naive Bayes estimates class probabilities under a simplifying conditional-independence assumption. Logistic Regression learns a linear decision boundary and outputs calibrated-looking probabilities. A Support Vector Machine seeks a large-margin boundary. Decision trees learn feature-based rules, and Random Forest aggregates many randomised trees.

Text data creates specific challenges. The vocabulary can be very large, each document contains only a small part of that vocabulary, and the same word can carry different meanings in different contexts. Negation, sarcasm, spelling errors, slang, code-mixing and domain shift make classification harder. A reliable experiment must represent these issues in its data and in its evaluation.

Table 4.1: Quick reference – text representation methods

Method	Representation	Strength	Limitation
Bag of Words	Term counts	Simple and interpretable	No word order unless n-grams are used
TF-IDF	Term frequency weighted by corpus rarity	Highlights informative terms	Still mostly lexical and sparse
Word embeddings	Dense vectors	Can represent semantic similarity	May lose some task-specific details
Transformer embeddings	Context-sensitive vectors	Captures contextual meaning	Higher compute and deployment cost

5. PREPROCESSING AND TF-IDF

Preprocessing makes text more consistent before feature extraction. Common steps include whitespace normalisation, removal or handling of markup, URL processing, case normalisation and tokenisation. Preprocessing must match the task: punctuation, emojis and repeated letters can carry useful sentiment information, so removing every non-alphanumeric symbol is not always appropriate.

Tokenisation divides text into words, sub-words or characters. Stop words are very common words that may add little value in some tasks, but terms such as “not” can reverse sentiment and should not be removed carelessly. Stemming reduces words to rough stems, whereas lemmatisation maps words to their dictionary base forms. Both may reduce the vocabulary but can also remove useful distinctions.

Bag of Words represents a document by the counts of its terms. It is simple and interpretable but ignores word order unless n-grams are included. TF-IDF weights a term by its frequency in a document and by its rarity across the corpus. A common smoothed inverse-document-frequency expression is

$$IDF(t) = ln((1 + N) / (1 + df(t))) + 1$$

where N is the number of documents and df(t) is the number of documents containing term t. The TF-IDF weight of term t in document d is tf(t, d) × IDF(t), and each document vector is then normalised to unit length so that long and short documents are comparable.

Worked example. Consider a tiny corpus of N = 3 reviews. The word “movie” appears in all three (df = 3), “good” in two (df = 2) and “not” in only one (df = 1). Table 5.1 shows the resulting IDF values. The rare word “not” receives the largest weight, while the ubiquitous word “movie” receives the smallest, which is exactly the behaviour TF-IDF is designed to provide.

Table 5.1: Illustrative IDF calculation for N = 3 documents

Term	df(t)	IDF(t) = ln((1 + 3)/(1 + df)) + 1	Value
movie	3	ln(4/4) + 1	1.000
good	2	ln(4/3) + 1	1.288
not	1	ln(4/2) + 1	1.693

N-grams are sequences of adjacent tokens. Unigrams capture individual words, while bigrams can capture phrases such as “not recommended”. N-grams may improve local context but increase dimensionality and sparsity. In this project the TF-IDF vectorizer uses unigrams and bigrams with a maximum feature limit.

Feature extraction must be fitted on training data only. Fitting a vectorizer on the entire dataset allows test-set vocabulary statistics to influence training and causes data leakage. A scikit-learn *Pipeline* keeps vectorisation and classification together, ensuring that the transformation learned from the training data is applied consistently to held-out examples.

6. ENSEMBLE LEARNING PRINCIPLES

Ensemble learning combines multiple models to create a final prediction. The models may be of the same type trained on different samples, or of different algorithm families trained on the same representation. Ensembles are most useful when their members are reasonably competent and make partly different errors. If every model makes the same mistakes, aggregation provides little improvement.

Bias and variance. Bias is error caused by overly restrictive assumptions, while variance describes sensitivity to the particular training sample. Bagging often reduces variance by averaging unstable learners. Boosting can improve a sequence of weak learners and may reduce bias. Voting can exploit complementary decision boundaries. These are common tendencies, not guarantees for every dataset.

Diversity. Diversity is important because different models can capture different aspects of the data. It can be created through bootstrap sampling, different algorithms, different feature subsets, different hyperparameters or different random seeds. Diversity alone is insufficient: weak models with unrelated errors may still perform poorly, so members must also be individually competent.

Aggregation. Aggregation can use class labels or class probabilities. Hard voting selects the most frequent label. Soft voting averages predicted probabilities and chooses the class with the highest average. Soft voting uses more information but depends on probability quality and calibration; a confident but poorly calibrated member can dominate the aggregate.

Design choices. Ensemble design requires decisions about base learners, feature representation, aggregation rule, model weights, tuning procedure and evaluation protocol. A fair experiment compares methods using identical splits and reports predictive scores together with runtime, memory use, interpretability and error patterns.

7. BAGGING

Bagging means **b**ootstrap **agg**regat**ing**. It repeatedly samples the training data with replacement to create bootstrap samples, fits a base learner to each sample and aggregates their predictions. Each learner sees a slightly different dataset, so its fitted model may differ from the others. Aggregation reduces the influence of any single sample-specific model.

The basic procedure is: begin with labelled training data; generate B bootstrap samples; train one learner on each sample; predict with all B learners for a new document; and return the majority label or average the predicted probabilities. For hard voting, the ensemble prediction can be written as the mode of $h_1(x)$, $h_2(x)$, …, $h_B(x)$.

A bootstrap sample of size n drawn from n examples leaves out each particular example with probability $(1 - 1/n)^n$, which approaches $e^{-1} \approx 0.368$ for large n. About 63.2% of the distinct training documents therefore appear in each sample, and the remaining 36.8% are “out-of-bag”. Out-of-bag examples can be used to estimate generalisation error without a separate validation set, although a final untouched test set is still recommended.

Decision trees are the most common bagging learners because they are unstable: small changes in the training examples may produce different splits. Aggregating many trees can reduce variance. In text tasks each tree can learn rules based on TF-IDF features, although deep trees can be expensive and may overfit high-dimensional sparse inputs.

Advantages of bagging include variance reduction for unstable learners, parallel training, a straightforward aggregation procedure and reduced dependence on a single fitted model. **Limitations** include additional memory and computation, limited benefit for stable learners such as linear models, and lower interpretability than a single simple classifier.

Useful parameters include the number of estimators, whether bootstrap sampling is used, the sample fraction, tree depth, the minimum samples per leaf, feature subsampling and the random seed. These should be tuned on validation data. Bagging may be useful for spam detection, sentiment classification

and topic categorisation, but linear models often remain strong baselines for sparse text.

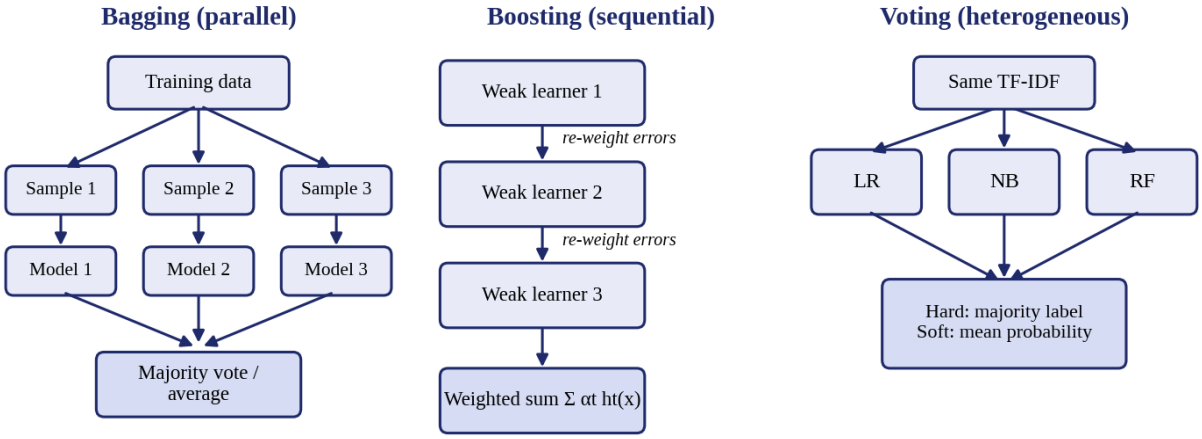


Figure 7.1: Conceptual comparison of bagging, boosting and voting

8. BOOSTING AND ADABOOST

Boosting trains weak learners sequentially. Each new learner is influenced by the errors made by the previous learners. Instead of fitting many independent models, boosting constructs a combined predictor in stages. This process can improve a weak learner's decision boundary, but it may also become sensitive to noise and incorrect labels.

AdaBoost, or Adaptive Boosting, begins with equal weights assigned to all training examples. A weak learner is fitted, its weighted error ε is calculated, and its contribution to the final model is determined by that error. Example weights are then adjusted so that misclassified examples receive more attention in the next round. Repeating the process creates a weighted combination of weak learners. In the classical binary formulation the learner weight and the weight update are

$$\alpha_t = \tfrac{1}{2} \ln((1 - \varepsilon_t) / \varepsilon_t), \; w_i \leftarrow w_i \cdot \exp(-\alpha_t y_i h_t(x_i)) / Z_t$$

where y_i and $h_t(x_i)$ take values in $\{-1, +1\}$ and Z_t is a normalising constant. The final prediction is the sign of the weighted sum $\Sigma \alpha_t h_t(x)$.

Worked example. Suppose 10 training documents start with weight 0.10 each, and the first stump misclassifies 2 of them, so ε = 0.20. Then α = ½ ln(0.8/0.2) = ½ ln 4 ≈ 0.693. Correctly classified documents are multiplied by $e^{-0.693} = 0.5$ and misclassified ones by $e^{0.693} = 2$. The unnormalised weights are 0.05 (eight documents) and 0.20 (two documents), which sum to 0.80. After normalisation the correct documents carry 0.0625 each and the misclassified ones 0.25 each; the two hard documents now hold half of the total weight, so the next stump is forced to concentrate on them (Figure 8.1).

A decision stump is a decision tree with a single split and is often used as an AdaBoost weak learner. Each stump captures a simple rule, and the ensemble combines many such rules. In text classification a split can depend on the value of a single token or n-gram feature. Sparse features and vocabulary size can affect the runtime of each round.

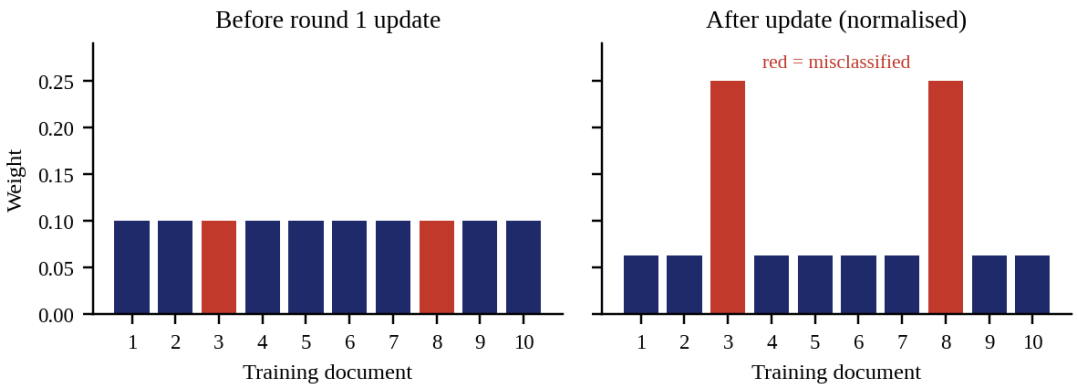


Figure 8.1: AdaBoost weight update for the worked example (ε = 0.20, α ≈ 0.693)

Boosting may perform well when the weak learners capture useful patterns and the labels are reliable. It can focus on difficult cases and turn a collection of simple learners into a stronger classifier. However, mislabelled examples or outliers can receive excessive emphasis, training is sequential, and results depend on the learning rate, the number of estimators and the weak-learner complexity.

Parameters to consider include the number of boosting rounds, the learning rate, the stump complexity and the random seed. A smaller learning rate may require more rounds. Model selection should use validation data rather than repeatedly consulting the test set. The exact scikit-learn parameter name for the base learner depends on the installed version.

9. VOTING CLASSIFIERS

A voting classifier combines predictions from multiple models. Unlike bagging, which usually repeats one base-learning method on bootstrap samples, voting often combines different algorithms. A text-classification ensemble may combine Logistic Regression, Multinomial Naive Bayes and Random Forest because they use different learning assumptions.

Hard voting counts class labels and selects the class with the most votes. If three models predict positive, negative and positive, the majority output is positive. This approach is simple and does not require probability estimates, but it ignores how confident each model is.

Soft voting combines predicted class probabilities. A weighted average can be written as

$$p(y \mid x) = \Sigma\, w_m\, p_m(y \mid x) / \Sigma\, w_m$$

where w_m are non-negative model weights. The class with the largest combined probability is selected. Soft voting can use confidence information, but probability calibration and comparability matter.

Worked example. Let Logistic Regression, Naive Bayes and Random Forest predict P(positive) = 0.70, 0.40 and 0.60 for a review. Hard voting gives positive, negative, positive, i.e. a positive majority. Soft voting with equal weights gives (0.70 + 0.40 + 0.60)/3 ≈ 0.567, also positive. If the weights are 2, 1 and 1 the average is (1.40 + 0.40 + 0.60)/4 = 0.60. The decisions agree here, but they can disagree when one member is extremely confident.

Logistic Regression learns a linear boundary and is often a strong text baseline. Multinomial Naive Bayes models term evidence using conditional probabilities. Random Forest captures nonlinear rules through

randomised trees. Their differences can produce complementary predictions, although the ensemble should only be retained if validation results justify its complexity.

Voting is modular and can combine different strengths, but it may be slower and larger than a single model. Adding a weak or redundant learner can reduce performance. Good practice includes inspecting each component's predictions, measuring error diversity, checking calibration and comparing the ensemble against every individual member.

Table 9.1: Summary comparison of bagging, boosting and voting

Feature	Bagging	Boosting	Voting
Training	Independent bootstrap models	Sequential learners	Different models can train independently
Main benefit	Often reduces variance	Learns from earlier errors	Combines complementary predictions
Concern	Memory and model size	Noise sensitivity	Calibration and added latency
Best practice	Tune tree complexity	Tune rounds and learning rate	Validate every member and the ensemble

10. COMPARISON OF ENSEMBLE METHODS

Bagging, boosting and voting all combine learners, but their mechanisms differ. Bagging changes the samples used to train members and aggregates them, often reducing variance. Boosting trains members sequentially and directs later learners toward earlier mistakes. Voting aggregates the outputs of selected models and may use either labels or probabilities.

Bagging members can usually be trained in parallel. Boosting rounds depend on earlier rounds, which limits parallel training. Voting components can often be trained independently, although prediction requires running every member. Their memory and latency costs therefore depend on the number and complexity of the component models.

Bagging may be relatively robust to individual unstable trees, while boosting can be sensitive to noisy labels because hard examples receive more attention. Voting depends strongly on the quality and diversity of its members. No method is always best; performance is shaped by data size, feature representation, class balance, noise and tuning.

Traditional text tasks often favour linear models with TF-IDF because the feature space is sparse and high-dimensional. This means that a voting ensemble should include a strong linear baseline, and tree ensembles should be evaluated rather than assumed to be superior. If scores are similar, simpler and faster models may be preferable.

Table 10.1: Practical trade-offs between the three ensemble strategies

Criterion	Bagging	Boosting	Voting
Parallel training	Yes, members are independent	No, rounds depend on each other	Yes, members are independent
Effect on bias/variance	Mainly lowers variance	Can lower bias, may raise variance	Depends on member diversity

Criterion	Bagging	Boosting	Voting
Sensitivity to label noise	Relatively low	Higher	Depends on members
Prediction cost	B model evaluations	T model evaluations	One evaluation per member
Interpretability	Low for many trees	Low to moderate for stumps	Moderate if members are linear

10.1 Related Ensemble Techniques

Random Forest is a specialised form of bagging in which each tree is also restricted to a random subset of features at every split. This extra randomisation decorrelates the trees, so averaging them reduces variance more effectively than plain bagging of fully grown trees. For text, the feature subset is drawn from thousands of sparse TF-IDF columns, which keeps individual trees diverse but makes training slower than for linear models.

Gradient boosting generalises the boosting idea: each new tree is fitted to the negative gradient of a loss function, so the ensemble gradually reduces any differentiable loss rather than only reweighting examples. Modern implementations add shrinkage, subsampling and regularisation. They are very strong on dense tabular data, but on sparse text a regularised linear model is frequently just as accurate and far cheaper.

Stacking trains a second-level model, often Logistic Regression, on the predictions of the first-level models. To avoid leakage, the first-level predictions given to the meta-learner must be produced out-of-fold using cross-validation. Stacking can learn how much to trust each member, but it is the most complex of these schemes and the easiest to evaluate incorrectly.

These techniques are outside the main experiment, but they show that bagging, boosting and voting are building blocks rather than isolated methods. A practitioner can combine them, for example by voting over a bagged tree model, a boosted model and a linear model, as long as every combination is validated against its simplest member.

11. SYSTEM ARCHITECTURE

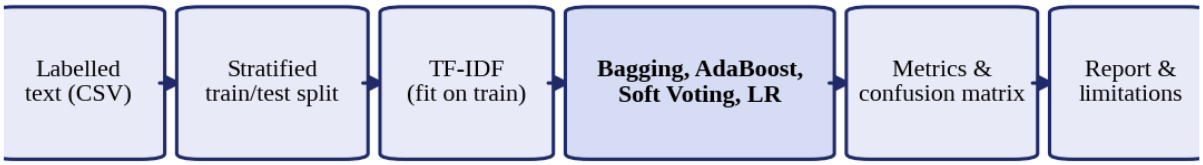


Figure 11.1: Overall system flow of the ensemble text-classification project

The system has five main stages: data input, feature extraction, model training, evaluation and result reporting (Figure 11.1). The CSV file contains a text column and a label column. A stratified split separates training and test examples. TF-IDF learns its vocabulary and weights from the training set and then transforms the test documents using the same fitted vectorizer.

Four pipelines are compared: bagging with decision trees, AdaBoost with decision stumps, soft voting with Logistic Regression, Naive Bayes and Random Forest, and a Logistic Regression baseline. Each pipeline is fitted on the training text and predicts labels for the held-out test text. Metrics are then computed from the predictions and the true labels.

Functional requirements include loading labelled data, splitting it reproducibly, extracting TF-IDF features, training models, calculating metrics, displaying plots and predicting sentiment for new text. Non-functional requirements include reproducibility, readable code, reasonable runtime, transparent limitations and portable files.

Python is used as the implementation language. pandas supports data loading and manipulation. scikit-learn provides vectorisation, pipelines, classifiers and evaluation metrics. Matplotlib creates the charts, while Jupyter Notebook combines explanation, code and output in a single document. A README records the dependencies and execution steps.

Table 11.1: Suggested public datasets for a rigorous extension

Dataset	Task	Scale / benefit
IMDb reviews	Sentiment	Large, established benchmark
SMS Spam Collection	Spam detection	Compact and easy to run
20 Newsgroups	Topic classification	Multiclass and varied topics
AG News	News categories	Useful for multiclass evaluation

12. DATASET AND EXPERIMENTAL DESIGN

The accompanying demonstration CSV contains short generated positive and negative sentences. It was created to make the project runnable without a dataset download. It is balanced by construction and includes repeated sentiment cues. These properties make it suitable for checking the pipeline but unsuitable for making strong claims about real-world generalisation.

The notebook uses a stratified 75% training and 25% test split with a fixed random seed. Stratification approximately preserves the class ratio in each partition. For stronger evidence, a validation set or cross-validation should be used for tuning, and the test set should be evaluated only after all model choices are complete.

Recommended public datasets include IMDb movie reviews for sentiment analysis, the SMS Spam Collection for spam filtering, 20 Newsgroups for topic classification and AG News for multiclass news categorisation. The dataset source, licence, label definitions, size and split rules should be recorded, and duplicate or near-duplicate documents should be checked before splitting.

Data leakage occurs when information from test examples influences model fitting or selection. It can occur through vectorizer fitting, feature selection, duplicate documents or repeated tuning against test results. A pipeline, a separation between tuning and final testing, and group-based or time-based splits (when records share a user, product, source or time period) are the main safeguards.

Table 12.1: Experimental protocol used in the project

Item	Setting
Task	Binary sentiment classification (positive / negative)
Data	Small generated teaching dataset (CSV with text and label columns)
Split	Stratified 75% training / 25% test, fixed random seed
Features	TF-IDF, unigrams and bigrams, at most 5,000 features
Models	Bagging, AdaBoost, soft voting, Logistic Regression baseline
Metrics	Accuracy, weighted precision, recall and F1, confusion matrix

Hyperparameter tuning follows a fixed protocol. Candidate values are searched only on the training portion using stratified five-fold cross-validation, the setting with the best mean validation F1-score is selected, the model is refitted on the full training set, and only then is the untouched test set scored once. Table 12.2 lists typical search ranges; they are starting points rather than recommendations for every corpus.

Table 12.2: Typical hyperparameter search ranges

Component	Parameter	Typical range
TF-IDF	ngram_range / min_df / max_features	(1,1)–(1,3) / 1–5 / 2,000–20,000
Bagging	n_estimators / max_depth	10–200 / 5–None
AdaBoost	n_estimators / learning_rate	25–500 / 0.05–1.0
Logistic Regression	C (inverse regularisation)	0.01–100 on a log scale
Soft voting	member weights	(1,1,1) and other small integer weights

13. IMPLEMENTATION DETAILS

The pipeline combines a TF-IDF vectorizer and a classifier so that the feature extractor is fitted only on the training text. Unigrams and bigrams are used, with a maximum of 5,000 features and English stop-word filtering. These settings are demonstration choices, not necessarily optimal values for every corpus.

Bagging uses 30 decision trees fitted on bootstrap samples. A fixed random seed supports repeatability, and parallel processing can speed up the independent learners. AdaBoost uses 50 decision stumps. The voting classifier combines Logistic Regression, Multinomial Naive Bayes and Random Forest using soft voting. Logistic Regression is also trained separately as a baseline.

The evaluation loop fits each pipeline, predicts the test labels and calculates accuracy, weighted precision, weighted recall and weighted F1-score. The results are placed in a comparison table and visualised in a bar chart. A confusion matrix is produced for the voting classifier to show correct and incorrect predictions by class.

An example prediction accepts a new sentence and passes it to the fitted voting pipeline. In practical systems a predicted label can be accompanied by a probability estimate, but a probability is not a guarantee of correctness. Low-confidence or out-of-domain inputs may need human review.

Software interfaces can vary across versions. In newer scikit-learn versions, BaggingClassifier and AdaBoostClassifier use the parameter name *estimator*; older versions may use *base_estimator*. The installed documentation should be checked and the package versions recorded so that the notebook remains reproducible. The complete code is listed in Appendix A.

Table 13.1: Model configuration used in the demonstration

Model	Base learner	Key settings
Bagging	Decision tree	30 estimators, bootstrap sampling, fixed seed
AdaBoost	Decision stump (depth 1)	50 rounds, default learning rate
Soft voting	LR + Multinomial NB + Random Forest	Equal weights, averaged probabilities
Baseline	Logistic Regression	Same TF-IDF features and split

14. EVALUATION METRICS

A confusion matrix compares true labels with predicted labels. In binary classification it counts true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN). These values reveal the kinds of mistakes a model makes, which can be hidden by overall accuracy.

Accuracy is (TP + TN) divided by all examples. It is easy to understand but can mislead on imbalanced datasets. Precision is TP/(TP + FP) and measures how many predicted positive cases are truly positive. Recall is TP/(TP + FN) and measures how many actual positive cases are identified.

The F1-score is the harmonic mean of precision and recall: F1 = 2 × precision × recall / (precision + recall). Macro F1 gives equal weight to each class, while weighted F1 weights the class scores by their support. Weighted scores reflect class frequencies but can hide poor results on rare classes, so per-class metrics are also important.

Table 14.1: Metric formulas and interpretation

Metric	Formula	Interpretation
Accuracy	(TP + TN) / total	Overall fraction correct
Precision	TP / (TP + FP)	Reliability of positive predictions
Recall	TP / (TP + FN)	Coverage of actual positives
F1-score	2PR / (P + R)	Balance of precision and recall

Illustrative example. Suppose a classifier produces TP = 45, FP = 5, FN = 10 and TN = 40 on 100 test documents (Table 14.2). Accuracy is 85/100 = 0.85, precision is 45/50 = 0.90, recall is 45/55 ≈ 0.82 and F1 = 90/105 ≈ 0.857. The model is therefore more reliable when it predicts positive than it is complete at finding positives. These numbers are invented to explain the formulas and are not results of this project.

Table 14.2: Confusion matrix layout with the illustrative counts

	Predicted positive	Predicted negative
Actual positive	TP = 45	FN = 10
Actual negative	FP = 5	TN = 40

Cross-validation evaluates a model across multiple splits and helps estimate variation. The mean and standard deviation should be reported where appropriate. If two models have similar scores, uncertainty should be considered before declaring a winner. For a rigorous study, statistical tests or bootstrap confidence intervals should be used where suitable.

Practical evaluation should also include runtime, memory use, prediction latency and model size. A small score improvement may not justify a large increase in inference cost. Metrics should reflect application needs: spam filtering may prioritise precision, while some screening tasks may place greater emphasis on recall.

15. RESULTS AND INTERPRETATION

The demonstration compares bagging, AdaBoost, soft voting and a Logistic Regression baseline on the same generated dataset and fixed split. In the run used to prepare the project files, all four models achieved 1.000 accuracy and 1.000 weighted precision, recall and F1-score. The notebook generates the exact metrics table and plots when it is executed.

Table 15.1: Result of the demonstration run (generated dataset)

Model	Accuracy	Precision	Recall	F1-score
Bagging	1.000	1.000	1.000	1.000
AdaBoost	1.000	1.000	1.000	1.000
Soft voting	1.000	1.000	1.000	1.000
Logistic Regression	1.000	1.000	1.000	1.000

These perfect scores are not evidence that the algorithms will perform perfectly on natural text. The data is small, formulaic and contains strong repeated sentiment cues. Training and test examples can share very similar patterns, making the classification task unusually easy. The scores verify the software workflow rather than establish a general ranking.

The result suggests that a linear baseline is sufficient for the current artificial dataset. There is no measured improvement from the added ensemble complexity in this split. A public benchmark may produce different scores and rankings because its vocabulary, writing style, ambiguity and noise are far more varied.

Error analysis should inspect false positives and false negatives and group them by text length, negation, spelling variation, slang, ambiguous sentiment and domain vocabulary. For multiclass tasks, confusion between related categories should be inspected. Such analysis helps to identify whether the model relies on superficial lexical cues or misses context.

Final reporting should include the dataset origin, class counts, split strategy, preprocessing, parameters, metrics and limitations. Statements such as “voting is the best model” should be avoided unless a representative validation protocol supports them. When results tie, simplicity, speed and maintainability should decide.

16. REAL-WORLD APPLICABILITY

Sentiment analysis labels reviews or comments as positive, negative or neutral. Ensembles can combine lexical models, but real sentiment includes sarcasm, negation, emojis, mixed opinions and domain-specific expressions. Evaluation data should represent these conditions.

Spam detection classifies messages as spam or legitimate. False positives can hide legitimate messages, while false negatives allow spam through. Precision and recall should be chosen according to the cost of these errors. Language changes over time, so temporal evaluation and monitoring can be valuable.

News categorisation assigns documents to topics such as sport, politics, business or science. It is a multiclass task, and metadata can accidentally reveal the label. A robust experiment should ensure that the classifier learns from the text content rather than from source-specific leakage.

Customer-support routing assigns incoming messages to billing, delivery, refunds, technical support or other categories. Misrouting increases response time. Systems can use confidence thresholds and send uncertain cases to human agents. Macro F1 helps assess performance across both common and less frequent categories.

Intent detection maps user messages to goals such as checking an order or requesting assistance. It must handle paraphrases and short ambiguous utterances. Content moderation and feedback analysis also use text classification, but they require attention to context, fairness, privacy and human oversight. In high-impact contexts, predictions should support rather than replace responsible review.

17. CHALLENGES AND LIMITATIONS

Dataset limitations are central to this project. A generated dataset lacks the diversity of real reviews and can produce unrealistically high scores. Public datasets may also be outdated, biased, duplicated or limited to one domain. Their provenance, licence, intended use and annotation process should be documented.

Overfitting occurs when a model learns training-specific details rather than general patterns. It can result from excessive complexity, small sample size, repeated test-set tuning or duplicates across splits. Cross-validation, regularisation, deduplication and a final untouched test set help reduce the risk.

Class imbalance can make accuracy misleading. Stratified splitting, class-wise metrics, macro F1, balanced accuracy and precision-recall curves can provide a clearer view. Resampling and class weights may help, but should be applied only within the training data.

Computational cost. Ensembles increase memory and computation. Bagging members can train in parallel, boosting is sequential, and voting must run each component at prediction time. Interpretability also decreases as model complexity grows. Feature importance is not a causal explanation, and explanation methods must be interpreted carefully.

Fairness and privacy. Text datasets may encode stereotypes, dialect differences or unequal coverage, and model quality can differ across language varieties and groups. Privacy is another concern because text may contain personal or confidential information. Repositories should contain only data that may legally and ethically be shared.

Concept drift. Language changes over time. New terms, events and product names can alter the relationship between text and labels. Monitoring, new annotations and periodic retraining may be needed. A random test split may not reveal this drift, so a future-time test can be useful.

Ethical considerations. Classifiers that sort text about people can affect what those people see, how quickly they are served and whether their messages are acted upon. Developers should document intended use, test performance separately for different language varieties, give users a way to contest decisions, and avoid deploying a model in a setting that its validation data does not represent. Automated labels should be treated as evidence for a human decision in high-impact cases, not as the decision itself.

18. FUTURE ENHANCEMENTS

First, evaluate public benchmark datasets such as IMDb, SMS Spam Collection, AG News or 20 Newsgroups. Documented splits should be used and an untouched test set preserved. This will make the comparison more meaningful and allow the results to be discussed in relation to prior work.

Second, use stratified cross-validation and tune hyperparameters such as the number of estimators, tree depth, learning rate, n-gram range, minimum document frequency, maximum features and regularisation strength. Tuning must not use test labels. Nested cross-validation may be appropriate for extensive model selection.

Third, compare additional baselines including Multinomial Naive Bayes, Complement Naive Bayes, linear SVM and regularised Logistic Regression. Linear models are strong baselines for sparse text, so ensemble complexity should be justified by a measured benefit.

Fourth, compare TF-IDF with pretrained transformer representations. Contextual encoders may capture semantic relationships and long-range context that lexical n-grams miss. Compute requirements, licence, latency and domain suitability must be recorded, and evaluation must remain leakage-aware.

Further improvements include probability calibration, validation-based threshold selection, class-wise error dashboards, model runtime measurement, explainability, experiment tracking and a simple demonstration interface. Deployment should include input validation, privacy controls, monitoring and a human-review path for uncertain predictions.

Finally, the experiment should be packaged for reuse: a fixed environment file, a documented random seed, saved model artefacts and a short README allow another student to repeat every table and figure exactly and to extend the comparison with new datasets.

19. CONCLUSION

This mini project surveyed bagging, boosting and voting classifiers for text classification. It described the need for preprocessing and TF-IDF, explained the learning principles of the three ensemble methods with worked numerical examples and outlined a common pipeline for training and evaluating them. A Logistic Regression baseline was included to check whether the additional complexity of ensembling produced a useful gain.

Bagging can reduce variance in unstable learners by aggregating models trained on bootstrap samples. Boosting builds a sequence of learners that focus on earlier mistakes. Voting combines model outputs and can exploit complementary strengths. Their success depends on data quality, feature representation, learner diversity, class balance and the cost of errors.

The included implementation provides a reproducible educational workflow. The generated dataset produced perfect scores for all models, but these results cannot establish real-world performance because the dataset is small and repetitive. A public benchmark, cross-validation, careful tuning and detailed error analysis are necessary before drawing a research conclusion.

The most appropriate classifier is not necessarily the most complex one. It is the model that provides reliable performance on representative unseen data while meeting the requirements for speed, interpretability, maintainability and resource use. The project can be extended into a stronger study by applying the same experimental controls to real-world datasets.

20. DEVELOPMENT METHODOLOGY

The project followed a structured, iterative methodology consisting of four phases: Requirements Analysis, System Design, Implementation and Testing. Each phase produced a defined output that was reviewed before the next phase began, and earlier phases were revisited whenever testing exposed a problem.

20.1 Requirements Analysis

The functional requirements were a labelled CSV input, a reproducible split, TF-IDF feature extraction, four comparable classifiers, standard metrics and plots, and prediction for new text. The non-functional requirements were reproducibility, readable code, reasonable runtime and transparent reporting of limitations.

20.2 System Design and Implementation

The design uses one scikit-learn Pipeline per model so that the vectorizer is always fitted on training text only. The implementation is a single Jupyter notebook using pandas, scikit-learn and Matplotlib, with a fixed random seed and recorded package versions. Testing consisted of running the notebook end to end on the generated dataset and checking that metrics, plots and example predictions were produced without errors.

Table 20.1: Methodology phases and activities

Phase	Activity	Output
1. Requirements Analysis	Identify tasks, data format and constraints	Requirements list
2. System Design	Pipeline design, choice of models and metrics	Design blueprint
3. Implementation	Coding in Python with scikit-learn	Notebook and code listing
4. Testing	End-to-end runs and manual checks	Verified working workflow

21. FEATURES OF THE PROJECT

The project includes the following key features, summarised in Table 21.1. Together they make the study easy to repeat, easy to extend and honest about what its results do and do not show.

Table 21.1: Project features summary

Feature	Description
Leakage-safe pipelines	TF-IDF is fitted on training text only inside each scikit-learn Pipeline.
Four compared models	Bagging, AdaBoost, soft voting and a Logistic Regression baseline under one protocol.
Reproducible workflow	Fixed random seed, stratified 75/25 split and recorded package versions.
Standard metrics	Accuracy, weighted precision, recall and F1-score, plus a confusion matrix.
Visual output	Comparison bar chart and confusion-matrix plot generated by the notebook.
Live prediction	Any new sentence can be classified by the fitted voting pipeline.
Transparent limitations	The report states clearly that scores on the generated dataset are not benchmarks.
Portable tools	Python, pandas, scikit-learn, Matplotlib and Jupyter on any major operating system.

REFERENCES

[1] Breiman, L. (1996). Bagging Predictors. *Machine Learning*, 24, 123–140. https://doi.org/10.1007/BF00058655

[2] Freund, Y., & Schapire, R. E. (1997). A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *Journal of Computer and System Sciences*, 55(1), 119–139. https://doi.org/10.1006/jcss.1997.1504

[3] Dietterich, T. G. (2000). Ensemble Methods in Machine Learning. *Multiple Classifier Systems*, 1–15. https://doi.org/10.1007/3-540-45014-9_1

[4] Rokach, L. (2010). Ensemble-based classifiers. *Artificial Intelligence Review*, 33, 1–39. https://doi.org/10.1007/s10462-009-9124-7

[5] Sebastiani, F. (2002). Machine learning in automated text categorization. *ACM Computing Surveys*, 34(1), 1–47. https://doi.org/10.1145/505282.505283

[6] Joachims, T. (1998). Text categorization with Support Vector Machines: Learning with many relevant features. *European Conference on Machine Learning*, 137–142. https://doi.org/10.1007/BFb0026683

[7] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. https://nlp.stanford.edu/IR-book/

[8] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

[9] Scikit-learn documentation, Ensemble methods: https://scikit-learn.org/stable/modules/ensemble.html

[10] Scikit-learn documentation, Text feature extraction: https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction

[11] UCI Machine Learning Repository, SMS Spam Collection: https://archive.ics.uci.edu/dataset/228/sms+spam+collection

[12] Maas, A. L., et al. (2011). Learning Word Vectors for Sentiment Analysis. *ACL-HLT*, 142–150. https://aclanthology.org/P11-1015/

APPENDIX A – PROGRAM SOURCE CODE

The following Python code reproduces the workflow described in Chapters 11 to 14. Save it next to a CSV file named *sentiment_data.csv* with the columns *text* and *label* and run it with Python 3 and scikit-learn 1.2 or later. The listing was executed on a small generated sentiment file to confirm that it runs without errors.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import (BaggingClassifier, AdaBoostClassifier,
                              RandomForestClassifier, VotingClassifier)
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score, precision_recall_fscore_support,
                             confusion_matrix, ConfusionMatrixDisplay)

SEED = 42
df = pd.read_csv("sentiment_data.csv")            # columns: text, label
X_train, X_test, y_train, y_test = train_test_split(
    df["text"], df["label"], test_size=0.25,
    stratify=df["label"], random_state=SEED)

def tfidf():
    return TfidfVectorizer(ngram_range=(1, 2), max_features=5000,
                           stop_words="english")

models = {
    "Bagging": BaggingClassifier(
        estimator=DecisionTreeClassifier(random_state=SEED),
        n_estimators=30, random_state=SEED, n_jobs=-1),
    "AdaBoost": AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1),
        n_estimators=50, random_state=SEED),
    "Soft Voting": VotingClassifier(
        estimators=[("lr", LogisticRegression(max_iter=1000)),
                    ("nb", MultinomialNB()),
                    ("rf", RandomForestClassifier(n_estimators=100,
                                                  random_state=SEED))],
        voting="soft"),
    "Logistic Regression": LogisticRegression(max_iter=1000),
}

rows, fitted = [], {}
for name, clf in models.items():
    pipe = Pipeline([("tfidf", tfidf()), ("clf", clf)])
    pipe.fit(X_train, y_train)                  # vectorizer fitted on train only
    pred = pipe.predict(X_test)
    p, r, f1, _ = precision_recall_fscore_support(
        y_test, pred, average="weighted", zero_division=0)
    rows.append({"Model": name, "Accuracy": accuracy_score(y_test, pred),
                 "Precision": p, "Recall": r, "F1": f1})
    fitted[name] = pipe

results = pd.DataFrame(rows).set_index("Model")
print(results.round(3))
results.plot(kind="bar", figsize=(8, 4), ylim=(0, 1.05))
plt.title("Model comparison"); plt.tight_layout(); plt.savefig("comparison.png")

vote = fitted["Soft Voting"]
cm = confusion_matrix(y_test, vote.predict(X_test))
ConfusionMatrixDisplay(cm).plot(); plt.savefig("confusion_matrix.png")
```

```
print(vote.predict(["The film was not good at all"]))
```

APPENDIX B – REPRODUCIBILITY CHECKLIST AND VIVA QUESTIONS

Reproducibility checklist. Document the dataset source and licence; check for duplicates; split before fitting features; record random seeds and package versions; tune only on training or validation data; report per-class metrics; include confusion matrices; record runtime; and state limitations and ethical considerations.

Viva questions. What is text classification? What is ensemble learning? Explain bagging and bootstrap sampling. How does AdaBoost update example weights? Differentiate hard and soft voting. Define TF-IDF. What is data leakage? Why can accuracy be misleading? What are precision, recall and F1-score? Why is a baseline necessary? Why are public benchmark datasets important?

Short model answers. *Bootstrap sampling:* drawing n documents with replacement from n, so roughly 63% of distinct documents appear in each sample. *AdaBoost weights:* misclassified documents are multiplied by exp(α) and correct ones by exp(−α), then all weights are normalised. *Hard versus soft voting:* hard voting counts labels, soft voting averages probabilities. *TF-IDF:* term frequency scaled by the rarity of the term across the corpus. *Data leakage:* any influence of test data on training or model selection. *Why a baseline:* to show whether the ensemble gives a real gain over a simple model.

How to run the project. Create a virtual environment, install pandas, scikit-learn and matplotlib, place the CSV file next to the script from Appendix A, run it, and compare the printed table with the saved bar chart and confusion matrix. Record the library versions printed by *pip freeze* in the README so that the experiment can be repeated later.

APPENDIX C – LIST OF SYMBOLS AND ABBREVIATIONS

Symbol / Abbreviation	Description
AI / ML	Artificial Intelligence / Machine Learning
BoW	Bag of Words
TF-IDF	Term Frequency–Inverse Document Frequency
N, df(t)	Number of documents; document frequency of term t
B	Number of bootstrap samples (bagging estimators)
ε, α	Weighted error and learner weight in AdaBoost
TP, TN, FP, FN	True/false positives and negatives
F1	Harmonic mean of precision and recall
LR / NB / RF	Logistic Regression / Naive Bayes / Random Forest
SVM	Support Vector Machine
CSV	Comma-Separated Values file